UNIT IV: INHERITANCE & POINTERS

Section A: Debugging Questions | Section B: Pseudocode Formulation

SECTION A — DEBUGGING QUESTIONS

Question 1 — Diamond problem in multipath hybrid inheritance

Topic: Hybrid inheritance, virtual base classes and ambiguity.

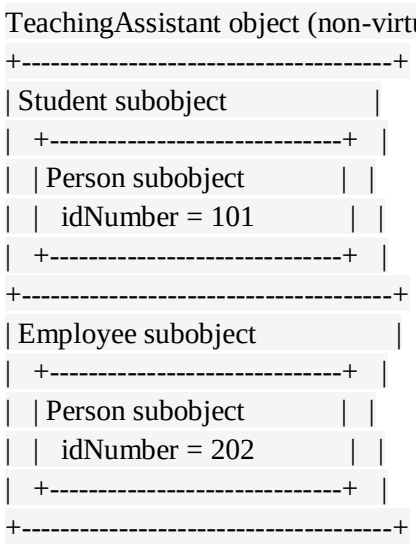
1. Exact compilation error at the ambiguous access

At the statement ta.idNumber (the question labels it Line 33), the compiler reports an ambiguity error, commonly phrased as:

```
error: request for member 'idNumber' is ambiguous
```

The exact wording varies by compiler. The expression is ambiguous because TeachingAssistant contains two distinct Person base-class subobjects.

2. Non-virtual memory layout and reason for duplication



Student and Employee each independently inherit Person using ordinary (non-virtual) inheritance. Therefore, the complete TeachingAssistant object has one Person subobject through Student and another through Employee. The compiler cannot infer which idNumber is intended. Explicitly selecting ta.Student::idNumber or ta.Employee::idNumber chooses different copies rather than creating one shared identity.

3. Corrected code using virtual base inheritance

```
#include <iostream>

class Person {
public:
    int idNumber;
    Person(int id = 0) : idNumber(id) {
        std::cout << "Person initialized: " << idNumber << std::endl;
    }
};

class Student : virtual public Person {
public:
    Student(int id) : Person(id) {}
};

class Employee : virtual public Person {
public:
    Employee(int id) : Person(id) {}
};

class TeachingAssistant : public Student, public Employee {
public:
    TeachingAssistant(int id)
        : Person(id), Student(id), Employee(id) {}
};

int main() {
    TeachingAssistant ta(101);
    std::cout << "TA ID: " << ta.idNumber << std::endl;
    return 0;
}
```

Both intermediate classes inherit Person virtually. The most-derived class, TeachingAssistant, contains one shared virtual Person subobject. The constructor now takes one shared ID, because there is only one Person::idNumber to initialize.

4. Constructor responsibility with a virtual base

The most-derived class is responsible for initializing a virtual base. Thus, when a TeachingAssistant is constructed, its initializer Person(id) initializes the shared Person subobject. The Person(id) initializers written in Student and Employee are used when objects of those classes are constructed as most-derived objects; when they are merely base subobjects of TeachingAssistant, those virtual-base initializers are ignored.

Expected output:

```
Person initialized: 101
TA ID: 101
```

Key correction: Use virtual public Person on both inheritance paths and initialize Person directly in the most-derived constructor.

Question 2 — Missing base constructor in derived initializer list

Topic: Derived constructors and constructor execution order.

1. Why the error occurs before Sensor's body

Device declares only Device(std::string); it has no zero-argument constructor. Because Sensor's constructor does not name Device in its member-initializer list, C++ tries to default-construct the base class before entering the Sensor constructor body. `error: no matching function for call to 'Device::Device()'`

2. Chronological constructor execution sequence

For a derived object, construction occurs in this order:

- Virtual base classes are initialized first (if present), by the most-derived constructor.
- Non-virtual direct base classes are initialized in the order they are declared in the class definition.
- Data members are initialized in the order they are declared in the class definition, not the order written in the initializer list.
- The derived-class constructor body executes last.

In this example there is one non-virtual base, Device, followed by Sensor's member samplingRate. Device must be initialized before Sensor's constructor body.

3. Corrected Sensor constructor

```
#include <iostream>
#include <string>

class Device {
protected:
    std::string macAddress;

public:
    Device(std::string mac) : macAddress(mac) {
        std::cout << "Device configured with MAC: "
                  << macAddress << std::endl;
    }
};

class Sensor : public Device {
private:
    int samplingRate;

public:
    Sensor(std::string mac, int rate)
        : Device(mac), samplingRate(rate) {}

    void display() const {
        std::cout << "MAC: " << macAddress
                  << ", Rate: " << samplingRate
                  << " Hz" << std::endl;
    }
};

int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
    return 0;
}
```

4. Expected terminal output

```
Device configured with MAC: 00:1A:2B:3C:4D:5E
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz
```

The base constructor prints first because Device is constructed before Sensor's body. The member initializer list : Device(mac), samplingRate(rate) initializes both the base and the derived data member correctly.

Question 3 — Base pointer member access and object slicing

Topic: Upcasting, static type checking and object slicing.

1. Why vPtr->engageTurbo() does not compile

The declared (static) type of vPtr is Vehicle*. The compiler checks member access using Vehicle's class declaration. Vehicle declares honk() and maxSpeed, but it does not declare engageTurbo(). The fact that the pointer's runtime address currently refers to a SportsCar object does not add SportsCar-only member names to the Vehicle interface.

```
error: 'class Vehicle' has no member named 'engageTurbo'
```

This is a compile-time member lookup issue, not a failure of the pointer to refer to the derived object. A virtual function can provide runtime dispatch only when the function is declared in the base-class interface and overridden in the derived class.

2. Definition and effect of object slicing

Object slicing occurs when a derived object is copied by value into a base-class object. In Vehicle plainVehicle = car;, only the Vehicle base portion is copied. The SportsCar-specific member turboEnabled is not part of plainVehicle and is discarded from that new object. The original car object remains unchanged; it is the copy into Vehicle that is sliced.

3. Correct downcast and ways to avoid slicing

Because the program knows that vPtr points to a SportsCar, it can use a checked downcast with dynamic_cast, provided Vehicle is polymorphic (has at least one virtual function). Alternatively, if the exact dynamic type is guaranteed by the program, static_cast can be used, but an incorrect static_cast downcast is unsafe.

```
// Make Vehicle polymorphic:
class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    virtual ~Vehicle() = default;
    void honk() { std::cout << "General Horn" << std::endl; }
};

// After vPtr has been assigned:
if (SportsCar* sportsPtr = dynamic_cast<SportsCar*>(vPtr)) {
    sportsPtr->engageTurbo();
}
```

To avoid slicing, keep a reference or pointer to the derived object instead of copying it into a base object:

```
SportsCar car(250, true);
Vehicle& vehicleRef = car;  // no copy; no slicing
Vehicle* vehiclePtr = &car; // no copy; no slicing
```

4. Corrected, compilable complete program

```
#include <iostream>

class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    virtual ~Vehicle() = default;

    void honk() {
        std::cout << "General Horn" << std::endl;
    }
};

class SportsCar : public Vehicle {
public:
    bool turboEnabled;
    SportsCar(int s, bool turbo)
        : Vehicle(s), turboEnabled(turbo) {}

    void engageTurbo() {
        if (turboEnabled)
            std::cout << "Turbo Engaged!" << std::endl;
        else
            std::cout << "Turbo Disabled" << std::endl;
    }
};

int main() {
    Vehicle* vPtr = new SportsCar(300, true);
    vPtr->honk();

    if (SportsCar* sportsPtr = dynamic_cast<SportsCar*>(vPtr)) {
        sportsPtr->engageTurbo();
    }

    SportsCar car(250, true);
    Vehicle& plainView = car; // reference: no slicing
    std::cout << "Max speed: " << plainView.maxSpeed << std::endl;

    delete vPtr;
    return 0;
}
```

Expected output:

```
General Horn
Turbo Engaged!
Max speed: 250
```

The virtual destructor ensures that deleting a derived object through a Vehicle pointer is safe. The downcast succeeds because vPtr points to a SportsCar. The reference plainView refers to the Vehicle portion of car without making a sliced Vehicle copy.

Question 4 — Incorrect return type in this-pointer method chaining

Topic: The this pointer and fluent interfaces.

1. Why the chained expression is rejected

setWidth() returns WindowBox*, so box.setWidth(800) is a pointer. The next expression uses the dot operator, .setHeight(600), but dot selects a member on an object. A pointer requires the arrow operator (->), or it must be dereferenced first. Therefore the expression as written is invalid.

```
error: request for member 'setHeight' in pointer type 'WindowBox*' (use '->' ?)
```

2. Difference between this and *this

Expression	Type / meaning	Use
this	Pointer to the current object; inside WindowBox member functions its type is WindowBox* (or const WindowBox* in a const member function).	Use -> to access members through it; return this returns a pointer.
*this	The object referred to by the this pointer; an lvalue referring to the current object.	Return by reference as WindowBox& with return *this; to continue dot-style chaining.

3. Correct setWidth declaration and return statement

```
WindowBox& setWidth(int width) {
    this->width = width;
    return *this;
}
```

Now setWidth returns a reference to the same WindowBox object. setHeight already returns WindowBox&, so the complete chain uses object references and dot syntax throughout.

4. Corrected complete code and expected output

```
#include <iostream>

class WindowBox {
private:
    int width;
    int height;

public:
    WindowBox() : width(0), height(0) {}

    WindowBox& setWidth(int width) {
        this->width = width;
        return *this;
    }

    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
    }

    void display() const {
        std::cout << "Window: " << width
                  << " x " << height << std::endl;
    }
};

int main() {
    WindowBox box;
    box.setWidth(800).setHeight(600);
    box.display();
    return 0;
}
```

```
Window: 800 x 600
```

The parameter names shadow the data-member names. this->width and this->height explicitly refer to the object's fields. Returning *this returns the current object by reference.

Question 5 — Private inheritance access violations and inaccessible base cast

Topic: Private inheritance and upcasting.

1. Accessibility of openPort() and closePort()

With class SecureChannel : private RawSocket, RawSocket's public and protected members become private members of SecureChannel for ordinary access through the derived class. They remain usable

inside SecureChannel's own member functions, which is why openPort(port) inside establishEncryptedTunnel() is legal. They are not publicly accessible through a SecureChannel object in main(), so channel.closePort() is rejected.

2. Why the upcast is inaccessible

The conversion from SecureChannel* to RawSocket* is an upcast, but private inheritance makes the RawSocket base inaccessible to unrelated client code. The conversion is allowed in appropriate contexts inside SecureChannel and its friends, but not in main(). In general, public inheritance permits an implicit public upcast from derived pointer/reference to base pointer/reference. Private inheritance blocks that conversion for outside clients; protected inheritance similarly restricts public access to the base conversion.

```
error: 'RawSocket' is an inaccessible base of 'SecureChannel'
```

3. Two ways to make closePort available

Approach A — selectively re-expose the base member: Add using RawSocket::closePort; in the public section of SecureChannel. This makes that inherited member publicly accessible through SecureChannel. It does not make the base conversion public.

Approach B — change the design: Use public inheritance if SecureChannel truly is a RawSocket and clients should be able to use it as one; or use composition by storing a RawSocket member and providing public wrapper methods. Composition is often preferable when the relationship is “has-a” rather than “is-a.”

4. Corrected code using Approach A

```
#include <iostream>

class RawSocket {
public:
    void openPort(int port) {
        std::cout << "Port " << port << " opened." << std::endl;
    }

    void closePort() {
        std::cout << "Port closed." << std::endl;
    }
};

class SecureChannel : private RawSocket {
public:
    using RawSocket::closePort;

    void establishEncryptedTunnel(int port) {
        openPort(port);
        std::cout << "Tunnel encrypted with TLS 1.3" << std::endl;
    }
};

int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);
    channel.closePort();

    // Still not permitted: private inheritance keeps the base
    // conversion inaccessible to outside code.
    // RawSocket* socketPtr = &channel;

    return 0;
}
```

Expected output:

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed
```

Important distinction: Approach A exposes closePort() but does not make RawSocket publicly accessible as a base. The upcast line remains invalid and is intentionally commented out.

SECTION B — PSEUDOCODE WRITING QUESTIONS

The pseudocode below follows the question bank's conventions: CLASS ... INHERITS, VIRTUAL inheritance, ABSTRACT CLASS, ABSTRACT METHOD, constructor delegation, pointers, NEW, arrow-style member access and RETURN REF THIS. Sample data is supplied where the question asks for a trace or output but does not prescribe every input.

Question 1 — University ERP hybrid hierarchy with virtual base classes

Topic: Hybrid inheritance, virtual base classes and diamond resolution.

1. Structured pseudocode

```
CLASS Person
    PRIVATE personID AS STRING
    PRIVATE fullName AS STRING
    PRIVATE email AS STRING

    CONSTRUCTOR Person(id, name, mail)
        personID <- id
        fullName <- name
        email <- mail
    END CONSTRUCTOR

    METHOD DisplayPerson()
        OUTPUT "Person ID: ", personID
        OUTPUT "Name: ", fullName
        OUTPUT "Email: ", email
    END METHOD
END CLASS


CLASS Student VIRTUAL INHERITS Person
    PROTECTED gpa AS REAL
    PROTECTED creditsEarned AS INTEGER

    CONSTRUCTOR Student(id, name, mail, grade, credits)
        BASE Person(id, name, mail)
        gpa <- grade
        creditsEarned <- credits
    END CONSTRUCTOR
END CLASS


CLASS Faculty VIRTUAL INHERITS Person
    PROTECTED academicDepartment AS STRING
    PROTECTED rankTitle AS STRING

    CONSTRUCTOR Faculty(id, name, mail, dept, rank)
        BASE Person(id, name, mail)
        academicDepartment <- dept
        rankTitle <- rank
    END CONSTRUCTOR
END CLASS


CLASS TeachingAssistant INHERITS Student, Faculty
```

```
    PRIVATE stipendAmount AS REAL
    PRIVATE assignedCourse AS STRING

    CONSTRUCTOR TeachingAssistant(id, name, mail,
                                  grade, credits, dept,
                                  rank, stipend, course)
        BASE Person(id, name, mail)
        BASE Student(id, name, mail, grade, credits)
        BASE Faculty(id, name, mail, dept, rank)
        stipendAmount <- stipend
        assignedCourse <- course
    END CONSTRUCTOR

    METHOD DisplayTARecord()
        OUTPUT "Person ID: ", personID
        OUTPUT "Name: ", fullName
        OUTPUT "GPA: ", gpa
        OUTPUT "Department: ", academicDepartment
        OUTPUT "Stipend: ", stipendAmount
        OUTPUT "Assigned Course: ", assignedCourse
    END METHOD
END CLASS
```

The word VIRTUAL on both Student and Faculty inheritance paths ensures that TeachingAssistant contains one shared Person subobject. The TeachingAssistant constructor, as the most-derived constructor, directly initializes Person. The intermediate constructor Person initializers are ignored for this virtual base when the object is a TeachingAssistant.

2. Sample instantiation trace and output

```
DECLARE ta AS TeachingAssistant
ta <- NEW TeachingAssistant(
    "P204", "Asha Raman", "asha@university.edu",
    3.85, 96, "Computer Science", "Teaching Fellow",
    18000.00, "CS204"
)
CALL ta.DisplayTARecord()
```

Step	Action
1	Initialize the one shared virtual Person subobject with P204, Asha Raman and asha@university.edu.
2	Construct Student base data: GPA = 3.85, creditsEarned = 96.
3	Construct Faculty base data: department = Computer Science, rank = Teaching Fellow.

4	Initialize TeachingAssistant data: stipend = 18000.00, course = CS204.
5	Call DisplayTARecord(); inherited Person fields are unambiguous because only one Person subobject exists.

Person ID: P204
Name: Asha Raman
GPA: 3.85
Department: Computer Science
Stipend: 18000
Assigned Course: CS204

Question 2 — Polymorphic payroll with abstract base class and base pointers
Topic: Abstract classes, pure virtual functions and polymorphism.

1. Structured pseudocode

```
ABSTRACT CLASS Employee
    PROTECTED employeeID AS STRING
    PROTECTED name AS STRING

    CONSTRUCTOR Employee(id, employeeName)
        employeeID <- id
        name <- employeeName
    END CONSTRUCTOR

    ABSTRACT METHOD CalculatePay() RETURNS REAL

    METHOD DisplayHeader()
        OUTPUT "Employee: ", employeeID, " - ", name
    END METHOD
END CLASS


CLASS SalariedEmployee INHERITS Employee
    PRIVATE monthlySalary AS REAL

    CONSTRUCTOR SalariedEmployee(id, employeeName, salary)
        BASE Employee(id, employeeName)
        monthlySalary <- salary
    END CONSTRUCTOR

    METHOD CalculatePay() RETURNS REAL
        RETURN monthlySalary
    END METHOD
END CLASS


CLASS HourlyEmployee INHERITS Employee
    PRIVATE hourlyWage AS REAL
    PRIVATE hoursWorked AS REAL

    CONSTRUCTOR HourlyEmployee(id, employeeName, wage, hours)
        BASE Employee(id, employeeName)
        hourlyWage <- wage
        hoursWorked <- hours
    END CONSTRUCTOR

    METHOD CalculatePay() RETURNS REAL
        IF hoursWorked <= 40 THEN
            RETURN hourlyWage * hoursWorked
```

```
        ELSE
            regularPay <- hourlyWage * 40
            overtimeHours <- hoursWorked - 40
            overtimePay <- overtimeHours * hourlyWage * 1.5
            RETURN regularPay + overtimePay
        END IF
    END METHOD
END CLASS


ALGORITHM Main
    DECLARE employees AS ARRAY OF POINTER TO Employee[2]
    DECLARE totalDisbursements AS REAL <- 0

    employees[0] <- NEW SalariedEmployee(
        "E101", "Mira", 8500
    )
    employees[1] <- NEW HourlyEmployee(
        "E102", "Dev", 50, 48
    )

    FOR i <- 0 TO 1
        CALL employees[i]->DisplayHeader()
        pay <- employees[i]->CalculatePay()
        OUTPUT "Pay: $", pay
        totalDisbursements <- totalDisbursements + pay
    END FOR

    OUTPUT "Total disbursements: $", totalDisbursements

    FOR i <- 0 TO 1
        DELETE employees[i]
    END FOR
END ALGORITHM
```

Employee is abstract because CalculatePay() is a pure virtual method. It defines a common interface but cannot be instantiated directly. Each concrete subclass overrides CalculatePay(). Calling it through an Employee pointer invokes the implementation belonging to the pointed-to object's runtime class.

2. Dry-run calculations

Employee	Calculation	Pay
SalariedEmployee	Monthly salary = 8,500	$8,500.00
HourlyEmployee	40 × $50 = $2,000 regular pay; 8 × $50 × 1.5 = $600 overtime pay	$2,600.00

Total	$8,500 + $2,600	$11,100.00

Hourly employee detail: hours above 40 are 48 − 40 = 8. Overtime rate is $50 × 1.5 = $75 per hour. Overtime pay is 8 × $75 = $600. Total hourly pay is $2,000 + $600 = $2,600.

Expected output (formatted to two decimal places)

```
Employee: E101 - Mira
Pay: $8500.00
Employee: E102 - Dev
Pay: $2600.00
Total disbursements: $11100.00
```

Question 3 — Composite canvas engine with member classes and this chaining

Topic: Composition and fluent method chaining.

1. Structured pseudocode

```
CLASS Point2D
    PRIVATE x AS REAL
    PRIVATE y AS REAL

    CONSTRUCTOR Point2D(xValue = 0.0, yValue = 0.0)
        x <- xValue
        y <- yValue
    END CONSTRUCTOR

    METHOD SetCoordinates(newX, newY)
        x <- newX
        y <- newY
    END METHOD

    METHOD GetX() RETURNS REAL
        RETURN x
    END METHOD

    METHOD GetY() RETURNS REAL
        RETURN y
    END METHOD
END CLASS


CLASS Circle
    PRIVATE centerPoint AS Point2D
    PRIVATE radius AS REAL

    CONSTRUCTOR Circle()
        centerPoint <- NEW Point2D(0.0, 0.0)
        radius <- 1.0
    END CONSTRUCTOR

    METHOD SetCenter(newX, newY) RETURNS REF Circle
        CALL centerPoint.SetCoordinates(newX, newY)
        RETURN REF THIS
    END METHOD

    METHOD SetRadius(r) RETURNS REF Circle
        IF r > 0 THEN
            radius <- r
        ELSE
            OUTPUT "Invalid radius: radius must be positive."
        END IF
```

```
        RETURN REF THIS
    END METHOD

    METHOD Draw()
        OUTPUT "Circle center: (", centerPoint.GetX(),
               ", ", centerPoint.GetY(), ")"
        OUTPUT "Radius: ", radius
    END METHOD
END CLASS
```

Circle has a Point2D member object called centerPoint. This is composition ("has-a"), not inheritance ("is-a"). The Point2D object is part of each Circle object's state. SetCenter and SetRadius return a reference to the same Circle object, represented by RETURN REF THIS, so further methods can be called using dot notation.

2. Trace of the required chained expression

```
DECLARE myCircle AS Circle
CALL myCircle.SetCenter(12.5, 30.0)
    .SetRadius(7.5)
    .Draw()
```

Step	Operation	State/result
1	Construct myCircle	centerPoint = (0.0, 0.0), radius = 1.0
2	SetCenter(12.5, 30.0)	centerPoint becomes (12.5, 30.0); returns reference to myCircle
3	SetRadius(7.5)	7.5 > 0, so radius becomes 7.5; returns reference to myCircle
4	Draw()	Prints the current center and radius

```
Circle center: (12.5, 30)
Radius: 7.5
```

The method calls do not create three Circle objects. Each setter modifies the same myCircle object and returns a reference to it. If a non-positive radius is passed, this pseudocode leaves the previous radius unchanged and prints the validation message.

Question 4 — Multi-level autonomous fleet telemetry with base pointers
Topic: Multilevel inheritance, constructor delegation and base pointers.

1. Structured pseudocode

```
CLASS Vehicle
    PROTECTED vehicleVIN AS STRING
    PROTECTED baseWeightKg AS REAL

    CONSTRUCTOR Vehicle(vin, weight)
        vehicleVIN <- vin
        baseWeightKg <- weight
    END CONSTRUCTOR

    METHOD DisplaySpecs()
        OUTPUT "VIN: ", vehicleVIN
        OUTPUT "Base weight (kg): ", baseWeightKg
    END METHOD
END CLASS


CLASS AutonomousVehicle INHERITS Vehicle
    PROTECTED lidarSensorID AS STRING
    PROTECTED firmwareVersion AS STRING

    CONSTRUCTOR AutonomousVehicle(vin, weight, lidar, firmware)
        BASE Vehicle(vin, weight)
        lidarSensorID <- lidar
        firmwareVersion <- firmware
    END CONSTRUCTOR
END CLASS


CLASS DeliveryDrone INHERITS AutonomousVehicle
    PRIVATE payloadCapacityKg AS REAL
    PRIVATE batteryChargePercent AS REAL

    CONSTRUCTOR DeliveryDrone(vin, weight, lidar, firmware,
                              payload, battery)
        BASE AutonomousVehicle(vin, weight, lidar, firmware)
        payloadCapacityKg <- payload
        batteryChargePercent <- battery
    END CONSTRUCTOR

    METHOD DisplayFlightStatus()
        CALL DisplaySpecs()
        OUTPUT "LiDAR sensor ID: ", lidarSensorID
        OUTPUT "Firmware version: ", firmwareVersion
        OUTPUT "Payload capacity (kg): ", payloadCapacityKg
```

```
        OUTPUT "Battery charge (%): ", batteryChargePercent
    END METHOD
END CLASS


ALGORITHM Main
    DECLARE vehiclePtr AS POINTER TO Vehicle
    DECLARE dronePtr AS POINTER TO DeliveryDrone

    dronePtr <- NEW DeliveryDrone(
        "VIN-DR-9081", 12.5, "LDR-44", "v2.6.1", 4.0, 87
    )

    vehiclePtr <- dronePtr       // implicit upcast
    CALL vehiclePtr->DisplaySpecs()

    dronePtr <- CAST vehiclePtr TO POINTER TO DeliveryDrone
    CALL dronePtr->DisplayFlightStatus()

    DELETE dronePtr
END ALGORITHM
```

The pointer conversion from DeliveryDrone* to Vehicle* is an upcast and is safe because Vehicle is an accessible public base in the described hierarchy. The reverse conversion is a downcast. It is valid here because the pointer is known to refer to a DeliveryDrone; in real C++, use dynamic_cast with a polymorphic base class when the dynamic type is not guaranteed.

2. Constructor invocation order

Order	Constructor/action	Reason
1	Vehicle(vin, weight)	Root base class is initialized first.
2	AutonomousVehicle(...)	Its Vehicle base is already initialized; then LiDAR and firmware members are initialized.
3	DeliveryDrone(...)	The most-derived constructor initializes drone-specific payload and battery members.
4	Main continues	The fully constructed DeliveryDrone is available through both derived and base pointers.

Constructor order is root to leaf: Vehicle → AutonomousVehicle → DeliveryDrone. Destruction occurs in reverse order: DeliveryDrone → AutonomousVehicle → Vehicle.

Sample output

```
VIN: VIN-DR-9081
Base weight (kg): 12.5
VIN: VIN-DR-9081
Base weight (kg): 12.5
LiDAR sensor ID: LDR-44
Firmware version: v2.6.1
Payload capacity (kg): 4
Battery charge (%): 87
```

The first two lines come from calling DisplaySpecs() through vehiclePtr. The full status call then invokes DisplaySpecs() again before printing the fields added by the derived classes.

Question 5 — Dynamic object pointer array and zero-copy sorting engine
Topic: Pointers to objects, arrow operator and pointer manipulation.

1. Structured pseudocode

```
CLASS Product
    PUBLIC sku AS STRING
    PUBLIC price AS REAL
    PUBLIC stockCount AS INTEGER

    CONSTRUCTOR Product(productSKU, productPrice, stock)
        sku <- productSKU
        price <- productPrice
        stockCount <- stock
    END CONSTRUCTOR

    METHOD PrintProduct()
        OUTPUT sku, " | $", price, " | Stock: ", stockCount
    END METHOD
END CLASS


ALGORITHM SortProductsByPrice(catalog, N)
    FOR i <- 0 TO N - 2
        FOR j <- 0 TO N - 2 - i
            // Read price through each pointer.
            IF catalog[j]->price > catalog[j + 1]->price THEN
                // Swap pointer values, not Product objects.
                tempPointer <- catalog[j]
                catalog[j] <- catalog[j + 1]
                catalog[j + 1] <- tempPointer
            END IF
        END FOR
    END FOR
END ALGORITHM


ALGORITHM Main
    DECLARE N AS INTEGER <- 3
    DECLARE catalog AS ARRAY OF POINTER TO Product[N]

    catalog[0] <- NEW Product("Laptop", 1200, 8)
    catalog[1] <- NEW Product("Mouse", 25, 50)
    catalog[2] <- NEW Product("Monitor", 350, 12)

    CALL SortProductsByPrice(catalog, N)

    OUTPUT "Products sorted by ascending price:"
    FOR i <- 0 TO N - 1
```

```
        CALL catalog[i]->PrintProduct()
    END FOR

    FOR i <- 0 TO N - 1
        DELETE catalog[i]
    END FOR
END ALGORITHM
```

The comparison uses catalog[j]->price, which accesses the Product object through a pointer. The swap exchanges only the pointer values stored in the array. The Product objects themselves are not copied or moved in memory. This is why it is described as pointer swapping or zero-copy sorting.

2. Dry-run trace for the three products

Initial array: index 0 → Laptop ($1200), index 1 → Mouse ($25), index 2 → Monitor ($350).

Pass / comparison	Comparison	Action	Array after action (by price)
Initial	—	—	Laptop 1200 | Mouse 25 | Monitor 350
Pass 1, j=0	1200 > 25	Swap pointers at 0 and 1	Mouse 25 | Laptop 1200 | Monitor 350
Pass 1, j=1	1200 > 350	Swap pointers at 1 and 2	Mouse 25 | Monitor 350 | Laptop 1200
Pass 2, j=0	25 > 350 is false	No swap	Mouse 25 | Monitor 350 | Laptop 1200

There are two pointer swaps. After the first pass the largest price, Laptop ($1200), has moved to the last position. The second pass confirms that Mouse ($25) and Monitor ($350) are already in ascending order.

Final output

```
Products sorted by ascending price:
Mouse | $25 | Stock: 50
Monitor | $350 | Stock: 12
Laptop | $1200 | Stock: 8
```

After printing, each pointer in the sorted array is deleted exactly once. Because sorting rearranges the pointers but does not duplicate or lose any pointer, every dynamically allocated Product is released once.

Final revision checklist

Concept	What to remember
Diamond problem	Ordinary multiple paths can create duplicate base subobjects; virtual inheritance shares one virtual

	base.
Virtual base constructor	The most-derived class initializes the virtual base.
Constructor order	Virtual bases → direct non-virtual bases → members → derived constructor body.
Base pointer	Member lookup uses the static pointer type; derived-only members need a suitable downcast.
Object slicing	Copying a derived object by value into a base object drops derived-only state.
this vs *this	this is a pointer; *this is the current object. Return a reference to *this for dot-style chaining.
Private inheritance	Base public members become private in the derived class; outside upcasting is inaccessible.
Abstract class	A class with a pure virtual function cannot be instantiated; concrete derived classes implement it.
Composition	A member object expresses a has-a relationship and is initialized as part of its containing object.
Pointer array sorting	Compare through ->, swap pointer variables, then delete each allocated object once.